Correction de la feuille de TP n°3

Structures de données

CHAÎNES DE CARACTÈRES

1 Exercice – Recherche d'un caractère

Q1. Écrire une fonction `e_recherche(ch)` qui prend en argument une chaîne de caractère et détermine si cette chaîne contient ou non le caractère "e" et, si c'est le cas, qui retourne le nombre de "e". Vous ne devez pas utiliser les fonctions préexistantes.

Q2. Essayer votre fonction sur la chaîne `"Le vieux fermier cueille sereinement des cerises et des fraises entre les arbres de son verger"`.

Q3. Quelle est la fonction préexistante qui joue le rôle de notre fonction `e_recherche(ch)` ?

Correction

Q1. On parcourt la chaîne caractère par caractère à l'aide d'un indice, en comptant les occurrences de `'e'` rencontrées ; s'il n'y en a aucune, on renvoie `False`, sinon on renvoie le compteur :

e_recherche.py

```
def e_recherche(ch):
    n = len(ch)
    nb = 0
    for i in range(n):
        if ch[i] == 'e':
            nb += 1
    if nb == 0:
        return False
    return nb
```

Q2.

console

```
In [1]: phrase = "Le vieux fermier
    cueille sereinement des cerises et
     des fraises entre les arbres de
    son verger"
In [2]: e_recherche(phrase)
Out[2]: 23
```

console

```
In [3]: phrase.count('e')
Out[3]: 23
```

Q3. La méthode préexistante `ch.count('e')` joue exactement ce rôle : elle renvoie directement le nombre d'occurrences de `'e'` dans `ch` (avec toutefois une petite différence : elle renvoie `0`, et non `False`, lorsque le caractère est absent).

2 Exercice – Palindrome

Un palindrome est un mot qui se lit aussi bien de gauche à droite que de droite à gauche. Par exemple : « elle », « ici », « ressasser », « radar », etc.

Écrivez une fonction `palindrome(mot)` qui retourne `1` si le mot donné en paramètre est un palindrome, et `0` dans le cas contraire.

Correction

Un mot est un palindrome lorsqu'il est égal à son propre renversement, qu'on obtient simplement par slicing avec un pas de -1 :

palindrome.py

```
def palindrome(mot):
    if mot == mot[::-1]:
        return 1
    else:
        return 0
```

console

```
In [1]: palindrome('elle')
Out[1]: 1

In [2]: palindrome('ressasser')
Out[2]: 1

In [3]: palindrome('kayak')
Out[3]: 1

In [4]: palindrome('python')
Out[4]: 0
```

3 Exercice – Caractère d'espacement

Écrivez une fonction `insertion(ch)` qui recopie `ch` (dans une nouvelle variable) en insérant des tirets entre les caractères. Ainsi, par exemple, `ch="Lyon Nice"` deviendra `ch_insertion = "L-y-o-n- -N-i-c-e"`.
Vous ne devez pas utiliser les fonctions préexistantes.

Correction

On parcourt la chaîne caractère par caractère, en reconstruisant une nouvelle chaîne à laquelle on ajoute un tiret après chaque caractère, sauf le dernier :

insertion.py

```
def insertion(ch):
    ch_insertion = ""
    for i in range(len(ch)):
        ch_insertion += ch[i]
        if i != len(ch)-1:
            ch_insertion += "-"
    return ch_insertion
```

console

```
In [1]: insertion("Lyon Nice")
Out[1]: 'L-y-o-n- -N-i-c-e'
```

Remarque. Une méthode préexistante permet d'obtenir directement ce résultat en une ligne : `"-".join(ch)`.

4 Exercice – **Rectangle**

Écrire une fonction qui prend en paramètres un caractère (par exemple `"X"`) et deux entiers, `largeur` et `hauteur`, et qui affiche un rectangle de dimension `largeur` × `hauteur`, entièrement rempli du caractère choisi.

console

```
In [1]: rectangle("X",4,3)
XXXX
XXXX
XXXX
```

Correction

Avec une double boucle `for` : la boucle extérieure parcourt les `hauteur` lignes du rectangle, la boucle intérieure construit chaque ligne caractère par caractère, en ajoutant `largeur` fois le caractère `car` :

rectangle.py

```
def rectangle(car, largeur, hauteur):
    for i in range(hauteur):
        ligne = ""
        for j in range(largeur):
            ligne += car
        print(ligne)
```

console

```
In [1]: rectangle("X",4,3)
XXXX
XXXX
XXXX
```

5 Exercice – **Sapin**

Proposer une fonction prenant un entier positif non nul `h` indiquant une hauteur et affichant, dans la sortie standard, un « sapin », constitué d'un motif de forme triangulaire fait du caractère `^` et d'un tronc constitué de deux `H`.

Par exemple, pour `h=4`, on souhaite obtenir le motif suivant :

console

```
   ^
   ^
  ^^^
 ^^^^^
^^^^^^^
   H
   H
```

Correction

Chaque étage k (de 1 à `h`) est constitué de $2k-1$ symboles `^`, précédés de `h-k` espaces pour centrer le triangle ; le tronc, lui, est formé de deux lignes contenant un seul `H`, précédé de `h-1` espaces :

sapin.py

```
def sapin(h):
    for k in range(1, h+1):
        print(' '*(h-k) + '^'*(2*k-1))
    for _ in range(2):
        print(' '*(h-1) + 'H')
```

console

```
In [1]: sapin(4)
   ^
  ^^^
 ^^^^^
^^^^^^^
   H
   H
```

DICTIONNAIRES

6 Exercice – **Polynôme**

Un polynôme peut être représenté par un dictionnaire, dont les clés sont les degrés des différents termes, et les valeurs les coefficients correspondants.

Par exemple, soit les deux polynômes $P_1 = 4$ et $P_2 = -X^5 + 2X^3 - 3$:

console
```
In [1]: P1 = {0:4}
In [2]: P2 = {0:-3, 3:2, 5:-1}
```

On adopte la convention que le polynôme nul est représenté par le dictionnaire vide `{}`.

Q1. Écrire une fonction `eval_poly(P, a)` qui prend en argument un polynôme `P`, représenté par un dictionnaire, et une valeur `a`, et qui renvoie la valeur $P(a)$.

console
```
In [3]: eval_poly(P1,3)
Out[3]: 4

In [4]: eval_poly(P2,-1)
Out[4]: -4

In [5]: eval_poly(P2,2)
Out[5]: -19
```

Q2. Écrire une fonction `derivee(P)` qui aura pour argument le dictionnaire `P` caractérisant un polynôme P, et qui renverra le dictionnaire associé au polynôme P', représentant sa dérivée.

console
```
In [6]: derivee(P1)
Out[6]: {}

In [7]: derivee(P2)
Out[7]: {2: 6, 4: -5}
```

Q3. Écrire alors une fonction `deriv_k(P,k)` qui aura pour argument le dictionnaire `P` caractérisant un polynôme P, et qui renverra le dictionnaire associé au polynôme $P^{(k)}$, représentant sa dérivée k-ième.

console
```
In [8]: deriv_k(P1,0)
Out[8]: {0: 4}

In [9]: deriv_k(P2,2)
Out[9]: {1: 12, 3: -20}

In [10]: deriv_k(P2,3)
Out[10]: {0: 12, 2: -60}
```

Correction

Q1. On parcourt les degrés présents dans le dictionnaire `P`, en sommant à chaque fois le terme $\texttt{coefficient} \times a^{\texttt{degre}}$:

eval_poly.py
```
1 def eval_poly(P, a):
2     resultat = 0
3     for degre in P:
4         resultat += P[degre] * a**
      degre
5     return resultat
```

Q2. Dériver un monôme $c \cdot X^d$ donne $d \cdot c \cdot X^{d-1}$; le terme constant ($d = 0$) disparaît. Il suffit donc de parcourir les degrés de `P`, en ignorant le degré 0, et de construire le nouveau dictionnaire en conséquence :

derivee.py
```
1 def derivee(P):
2     resultat = {}
3     for degre in P:
4         if degre > 0:
5             resultat[degre-1] =
      degre * P[degre]
6     return resultat
```

Q3. Il suffit d'appliquer `derivee` k fois de suite. Pour ne pas modifier `P` (dictionnaire mutable, passé par référence), on part d'une **copie** de `P` plutôt que de `P` lui-même :

deriv_k.py
```
1 def deriv_k(P, k):
2     resultat = P.copy()
3     for _ in range(k):
4         resultat = derivee(resultat)
5     return resultat
```

On vérifie bien que `P2` n'a pas été modifié par les appels successifs à `deriv_k` : c'est précisément grâce au `.copy()` initial, qui évite de travailler directement sur la référence transmise en argument.

LISTES

Dans toute la suite, il est interdit d'utiliser les fonctions ou opérateurs préexistants qui feraient le travail à notre place (l'opérateur `in`, les méthodes `.index()`, `.count()`, les fonctions `max`, `sum`, etc.) : le but de ces exercices est précisément de les réimplémenter soi-même.

7 Exercice – **Appartenance à une liste**

Écrire une fonction testant l'appartenance d'un objet

à une liste. On en proposera une version à l'aide d'une boucle `for`, et une autre à l'aide d'une boucle `while`.

appartient.py

```
def appartient(x, L):
    ...
```

console

```
In [1]: appartient(42, [5, 12, 17, 42])
Out[1]: True

In [2]: appartient(24, [5, 12, 17, 42])
Out[2]: False
```

Correction

Version avec une boucle `for`.

On parcourt les éléments de la liste un à un ; dès qu'on trouve `x`, on peut renvoyer `True` immédiatement (inutile de continuer à chercher). Si la boucle se termine sans avoir rien trouvé, c'est que `x` n'est pas présent :

appartient_for.py

```
def appartient(x, L):
    for y in L:
        if y == x:
            return True
    return False
```

Version avec une boucle `while`.

On parcourt cette fois la liste à l'aide d'un indice `i`, qu'on incrémente tant qu'on n'a pas trouvé `x` et qu'on n'a pas atteint la fin de la liste :

appartient_while.py

```
def appartient(x, L):
    i = 0
    n = len(L)
    while i < n:
        if L[i] == x:
            return True
        i += 1
    return False
```

8 Exercice – Positions d'un objet dans une liste

Écrire une fonction renvoyant la liste (éventuellement vide) de toutes les positions auxquelles un objet apparaît dans une liste.

positions.py

```
def positions(x, L):
    ...
```

console

```
In [1]: positions(42, [12, 17, 42, 5])
Out[1]: [2]

In [2]: positions(24, [12, 17, 42, 5])
Out[2]: []

In [3]: positions(42, [42, 12, 17, 42, 5])
Out[3]: [0, 3]
```

Correction

On parcourt la liste à l'aide d'un indice `i`, en ajoutant cet indice à une liste résultat chaque fois que `L[i]` vaut `x` :

positions.py

```
def positions(x, L):
    resultat = []
    for i in range(len(L)):
        if L[i] == x:
            resultat.append(i)
    return resultat
```

9 Exercice – Maximum d'une liste

Q1. Pour une liste **dont les éléments sont des entiers**, écrire une fonction renvoyant le *maximum* des éléments de cette liste. Écrire de même une fonction renvoyant la *position* de ce maximum.

console

```
In [1]: L1 = [5, 12, 42, 7, 42, 3]
In [2]: maximum(L1), position_maximum(
    L1)
Out[2]: (42, 2)
```

Q2. Que se passe-t-il lorsque le maximum apparaît plusieurs fois dans la liste ? Comment modifier le code pour changer ce comportement si besoin ?

Correction

Q1. On initialise le maximum (resp. sa position) avec le premier élément de la liste, puis on parcourt les éléments suivants en mettant à jour dès qu'on trouve mieux :

maximum.py

```
def maximum(L):
    m = L[0]
    for x in L:
        if x > m:
            m = x
    return m

def position_maximum(L):
    imax = 0
    for i in range(1, len(L)):
        if L[i] > L[imax]:
            imax = i
    return imax
```

console

```
In [1]: L1 = [5, 12, 42, 7, 42, 3]
In [2]: maximum(L1), position_maximum(
    L1)
Out[2]: (42, 2)
```

Q2. Ici, la comparaison stricte `L[i] > L[imax]` fait que `position_maximum` renvoie la **première** position où le maximum est atteint (indice 2, bien que 42 apparaisse aussi en position 4) : une fois `imax` fixé sur cette première occurrence, une valeur seulement égale ne le remplace plus. Si l'on souhaite au contraire récupérer la **dernière** occurrence du maximum, il suffit de remplacer la comparaison stricte par une comparaison large, `L[i] >= L[imax]` :

console

```
In [3]: position_maximum_dernier(L1)
Out[3]: 4
```

10 Exercice – Sommes cumulées

Q1. Écrire une fonction prenant en entrée une liste de nombres, et renvoyant la liste des sommes cumulées (c'est-à-dire, à chaque position, la somme de tous les termes qui la précèdent, elle incluse) :

console

```
In [1]: L2 = [4, 6, 2, 8, 5, 3, 9, 1,
    7]
In [2]: sommes_cumulees(tL)
Out[2]: [4, 10, 12, 20, 25, 28, 37, 38,
     45]
```

Q2. Évaluer le nombre d'additions réalisées par votre fonction, en fonction de la longueur n de la liste. Si ce nombre est de l'ordre de n^2, essayer de réécrire la fonction pour arriver à un nombre d'additions de l'ordre de n.

Correction

Q1. Une première idée consiste, pour chaque position i, à ressommer depuis le début tous les termes qui la précèdent :

sommes_cumulees_v1.py

```
def sommes_cumulees(L):
    n = len(L)
    s = []
    for i in range(n):
        somme = 0
        for j in range(i+1):
            somme += L[j]
        s.append(somme)
    return s
```

console

```
In [1]: L2 = [4, 6, 2, 8, 5, 3, 9, 1,
    7]
In [2]: sommes_cumulees(L2)
Out[2]: [4, 10, 12, 20, 25, 28, 37, 38,
     45]
```

Q2. À la position i (indices de 0 à $n-1$), cette version effectue $i+1$ additions ; le nombre total d'additions vaut donc $1+2+\cdots+n = \frac{n(n+1)}{2}$, de l'ordre de n^2 : chaque somme est en effet recalculée entièrement depuis le début, ce qui est redondant.

On peut éviter cette redondance en conservant, au fil du parcours, la valeur de la somme déjà calculée, et en se contentant d'y ajouter le terme courant :

sommes_cumulees_v2.py

```
def sommes_cumulees(L):
    s = []
    total = 0
    for x in L:
        total += x
        s.append(total)
    return s
```

console

```
In [3]: sommes_cumulees(L2)
Out[3]: [4, 10, 12, 20, 25, 28, 37, 38,
     45]
```

Cette seconde version n'effectue plus qu'une

seule addition par élément de la liste, soit n additions au total, de l'ordre de n.

11 Exercice – Plus longue sous-liste croissante

On appelle *sous-liste* d'une liste `L` (de longueur `n`) toute liste de la forme `[L[n0],L[n1],...,L[nk]]`, où $0 \leqslant n_0 < n_1 < \cdots < n_k < n$. Elle est dite *croissante* si de plus `L[n0]` $\leqslant$ `L[n1]` $\leqslant \cdots \leqslant$ `L[nk]` ; par convention, une liste réduite à un seul élément est toujours croissante.

On cherche ici un algorithme efficace pour déterminer la plus longue sous-liste croissante d'une liste donnée. L'idée consiste à construire une nouvelle liste `M` de même longueur que `L`, où `M[i]` désigne la longueur de la plus longue sous-liste croissante de `L` **se terminant** en position `i` :

- ▷ si aucun terme avant `L[i]` ne lui est inférieur ou égal, alors `M[i] = 1` (la sous-liste réduite à `L[i]` seul) ;
- ▷ sinon, `M[i]` vaut $1 + \max($`M[j]`$)$, le maximum portant sur tous les indices `j < i` tels que `L[j] <= L[i]`.

Q1. Comment obtenir la longueur d'une plus longue sous-liste croissante de `L` à partir de `M` ?

Q2. Écrire une fonction `maxLongSL` qui renvoie la longueur d'une plus longue sous-liste croissante d'une liste `L` donné en entrée.

On s'autorise à utiliser la fonction `max` sur les listes.

Q3. Écrire une fonction `maxSL` renvoyant une sous-liste de taille maximale de la liste `L` donné en argument.

Correction

Q1. La longueur d'une plus longue sous-liste croissante de `L` est simplement le **maximum** des valeurs de `M` : en effet, toute sous-liste croissante se termine bien quelque part, à un indice `i` donné, et `M[i]` est précisément la longueur de la meilleure sous-liste croissante se terminant à cet indice.

Q2. Il suffit de calculer `M` terme à terme selon la relation de récurrence donnée dans l'énoncé, puis de renvoyer son maximum :

maxLongSL.py

```
def maxLongSL(L):
    n = len(L)
    M = [1]*n
    for i in range(n):
        for j in range(i):
            if L[j] <= L[i] and M[j]+1 > M[i]:
                M[i] = M[j]+1
    return max(M)
```

console

```
In [1]: L = [3, 1, 4, 1, 5, 9, 2, 6]
In [2]: maxLongSL(L)
Out[2]: 4
```

Q3. Pour reconstruire une sous-liste (et pas seulement sa longueur), on retient en plus, pour chaque indice `i`, l'indice `j` qui a permis d'atteindre le maximum dans la relation de récurrence (ou `-1` s'il n'y en a pas) : c'est le *prédécesseur* de `i` dans la sous-liste optimale. Il ne reste plus qu'à repérer l'indice où `M` est maximal, puis à remonter la chaîne des prédécesseurs jusqu'à `-1` :

maxSL.py

```
def maxSL(L):
    n = len(L)
    M = [1]*n
    pred = [-1]*n
    for i in range(n):
        for j in range(i):
            if L[j] <= L[i] and M[j]+1 > M[i]:
                M[i] = M[j]+1
                pred[i] = j

    imax = 0
    for i in range(1, n):
        if M[i] > M[imax]:
            imax = i

    sous_liste = []
    i = imax
    while i != -1:
        sous_liste.append(L[i])
        i = pred[i]
    sous_liste.reverse()
    return sous_liste
```

console

```
In [3]: maxSL(L)
Out[3]: [3, 4, 5, 9]
```